🎥 YOUTUBE VIDEO SCRIPT

“Enterprise HR Attendance Automation Using Python | Real-World Project”

🎬 INTRO (0:00 – 0:40)

🎙️ **You say:**

“In this video, we’ll build a **real enterprise-level HR attendance automation system using Python**.”

“This script:

- Reads attendance from Excel
- Finds employees working less than 8 hours
- Sends alert emails to HR and Managers
- Sends warning emails to employees
- Generates a monthly summary report
- Uses logging and error handling like real companies”

“This is NOT a toy project. This is **resume-level, production-ready code**.”

🧠 PREREQUISITES (0:40 – 1:10)

🎙️ **You say:**

“To understand this video, you should know:

- Python basics
- Functions
- Loops
- Basic Excel
  I’ll explain everything else step by step.”

📁 PROJECT OVERVIEW (1:10 – 1:50)

🎙️ **You say:**

“Let’s understand the workflow first.”

📌 **Show flow on screen:**

Excel → Python → Filter → Email → Report → Logs

“Everything starts from Excel and ends with automated emails and reports.”

📦 **IMPORTS SECTION (1:50 – 3:30)**

📌 **Show code**

```
import smtplib
import pandas as pd
import os
import logging
import re
from email.message import EmailMessage
from pathlib import Path
from dotenv import load_dotenv
```

🎙️ **Explain:**

- smtplib → send emails
- pandas → read Excel
- os → environment variables
- logging → professional logs
- re → email validation using regex
- EmailMessage → structured emails
- Path → safe file handling
- dotenv → load .env file

“These libraries together make our script enterprise-ready.”

📜 **LOGGING CONFIGURATION (3:30 – 5:30)**

📌 **Show code**

logger = logging.getLogger()

logger.setLevel(logging.INFO)

🎙️ **Explain:**

“Instead of using print statements, we use logging.”

“Logs help us:

- Debug issues
- Track execution
- Maintain production systems”

📌 **File & Console Logging**

file_handler = logging.FileHandler("attendance.log", encoding="utf-8")

console_handler = logging.StreamHandler()

🎙️ **Explain:**

“Logs are written both to:

- A file (for history)
- Console (for live monitoring)”

🔐 **ENVIRONMENT VARIABLES (5:30 – 6:40)**

📌 **Show code**

load_dotenv()

SENDER_EMAIL = os.getenv("SENDER_EMAIL")

SENDER_PASSWORD = os.getenv("SENDER_PASSWORD")

🎙️ **Explain:**

“Never hardcode passwords.”

“We store email credentials securely in a .env file.”

📌 Example .env:

SENDER_EMAIL=example@gmail.com

SENDER_PASSWORD=app_password

📧 EMAIL CONFIGURATION (6:40 – 7:30)

SMTP_SERVER = "smtp.gmail.com"

SMTP_PORT = 587

🎙️ **Explain:**

“We use Gmail SMTP with TLS encryption.”

“This is industry standard.”

📁 FILE CONFIGURATION (7:30 – 8:30)

EXCEL_FILE = "data.xlsx"

SHEET_NAME = "Attendance"

MIN_WORK_HOURS = 8

🎙️ **Explain:**

“These are configuration values.”
“If business rules change, we update them here.”

✅ EMAIL VALIDATION FUNCTION (8:30 – 11:30)

📌 **Show code**

```
def is_valid_email(email):
    pattern = r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$"
    return isinstance(email, str) and re.match(pattern, email)
```

🎙️ **Explain:**

“This function checks whether an email is valid.”

“Regex ensures:

- Correct format
- No spaces
- Proper domain”

📌 **Validate Email List**

```
def validate_email_list(email_list, label):
   for email in email_list:
      if not is_valid_email(email):
         raise ValueError(...)
```

🎙️ **Explain:**

“If even ONE email is wrong, we stop the program immediately.”

“This follows the **fail-fast principle**.”

📊 **READING EXCEL FILE (11:30 – 14:00)**

📌 **Show code**

```
df = pd.read_excel(EXCEL_FILE, sheet_name=SHEET_NAME)
```

🎙️ **Explain:**

“We read attendance data into a DataFrame.”

📌 **Cleaning Email Column**

```
df["Email"] = (
   df["Email"].astype(str)
   .str.strip()
   .str.replace(r"\s+", "", regex=True)
   .str.lower()
)
```

🎙️ **Explain:**

“This avoids email delivery failures caused by:

- Spaces
- Capital letters

- Invalid formats”

⏱️ FILTERING SHORT HOURS (14:00 – 15:00)

df[df["Total Hours"] < MIN_WORK_HOURS]

🎙️ **Explain:**

“We filter employees who worked less than 8 hours.”

“This is the core business logic.”

🎨 HTML TABLE WITH HIGHLIGHT (15:00 – 16:30)

df.style.map(color_hours, subset=["Total Hours"])

🎙️ **Explain:**

“We convert the DataFrame into an HTML table.”

“Short hours are highlighted in RED so HR can quickly notice.”

📩 EMAIL SENDING FUNCTION (16:30 – 19:00)

📌 **Show code**

with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:

server.starttls()

server.login(...)

server.send_message(...)

🎙️ **Explain:**

“This function:

- Connects to SMTP server
- Encrypts connection
- Logs in
- Sends email”

“This is how real email automation works.”

👤 **EMPLOYEE ALERT EMAIL (19:00 – 20:30)**

🎙️ **Explain:**

“Each employee gets a **personalized warning email**.”

“This improves accountability.”

📈 **MONTHLY SUMMARY REPORT (20:30 – 22:00)**

df.groupby("Name")["Total Hours"].mean()

🎙️ **Explain:**

“We calculate average monthly hours for each employee.”

“The report is saved automatically to Excel.”

🚀 **MAIN AUTOMATION FLOW (22:00 – 24:00)**

🎙️ **Explain flow verbally:**

1. Validate HR emails
2. Read attendance
3. Filter short hours
4. Send HR report
5. Send employee alerts
6. Generate summary

“One function controls the entire automation.”

▶️ **SCRIPT EXECUTION (24:00 – 24:30)**

```
if __name__ == "__main__":
    run_automation()
```

🎙️ **Explain:**

“This ensures the script runs only when executed directly.”

🏁 **OUTRO (24:30 – 25:00)**

🎙️ **You say:**

“This is how enterprise Python automation is built.”

“If you understood this, you’re already ahead of many developers.”

“Like, Subscribe, and Comment if you want:

- Docker version
- Cloud deployment
- Interview questions
- Resume project description”

✅ **BONUS: VIDEO TITLE IDEAS**

- **Enterprise HR Automation Using Python**
- **Real-World Python Project with Email Automation**
- **Python Attendance System (Excel + Email)**
- **Production-Ready Python Automation**